

**TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA CÔNG NGHỆ THÔNG TIN**



BÀI TIỂU LUẬN

Đề Tài:

**Xây dựng đồng hồ thời gian thực với
ESP32 và NTP**

**Giáo viên hướng dẫn:
Thầy Võ Việt Dũng**

Môn : Phát triển ứng dụng IoT

**Sinh viên thực hiện :
Dương Hữu Tiến**

Nhóm : 3

Năm học : 2025 – 2026

Huế, 4/2026

MỤC LỤC

DANH MỤC CÁC TỪ VIẾT TẮT	2
LỜI CẢM ƠN.....	3
PHẦN 1: MỞ ĐẦU	3
1.1. Bối cảnh và tầm quan trọng của đề tài	3
1.2. Lý do chọn đề tài.....	3
1.3. Mục tiêu và phạm vi	4
1.3.1.Mục tiêu đề tài:.....	4
1.3.2 Phạm vi nghiên cứu:	4
1.4. Ý nghĩa của đề tài	4
PHẦN 2: PHẦN NỘI DUNG	4
2.1.Nội dung của đề tài	4
2.1.1 Tổng quan về hệ thống:.....	4
2.1.2.Thiết kế hệ thống:.....	5
2.1.2.1 Sơ đồ logic:.....	5
2.1.2.2. Các linh kiện và chức năng:.....	5
2.1.2.3. Nguyên lý hoạt động:	5
2.1.2.4. Sơ đồ mạch:	5
2.1.3. Tổng quan về công nghệ liên quan.....	6
2.1.3.1. Giao thức NTP (Network Time Protocol)	6
2.1.3.2. Giao thức I2C (Inter-Integrated Circuit).....	7
2.1.3.3. Telegram API.....	7
2.1.4. So sánh với các giải pháp khác.....	7
2.2. Cài đặt hệ thống.....	7
2.3. Kết quả thực hiện	9
PHẦN 3: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	10
3.1. Kết luận.	10
3.2. Nhận xét.....	11
3.3. Đánh giá.....	11
3.4. Kiến nghị	11
TÀI LIỆU THAM KHẢO	12

DANH MỤC CÁC TỪ VIẾT TẮT

- API: Application Programming Interface (Giao diện lập trình ứng dụng)
- GMT: Greenwich Mean Time (Giờ trung bình Greenwich / Múi giờ chuẩn)
- GND: Ground (Chân nối đất / Cực âm)
- HTTP: HyperText Transfer Protocol (Giao thức truyền tải siêu văn bản)
- HTTPS: HyperText Transfer Protocol Secure (Giao thức truyền tải siêu văn bản có bảo mật)
- I2C: Inter-Integrated Circuit (Chuẩn giao tiếp nối tiếp đồng bộ)
- IDE: Integrated Development Environment (Môi trường phát triển tích hợp)
- IoT: Internet of Things (Internet vạn vật)
- NTP: Network Time Protocol (Giao thức đồng bộ thời gian qua mạng)
- OLED: Organic Light-Emitting Diode (Công nghệ Đi-ốt phát quang hữu cơ)
- PCB: Printed Circuit Board (Bảng mạch in)
- RTC: Real-Time Clock (Đồng hồ / Mạch thời gian thực)
- SCL: Serial Clock (Chân tín hiệu xung nhịp - Giao tiếp I2C)
- SDA: Serial Data (Chân tín hiệu dữ liệu - Giao tiếp I2C)
- VCC: Voltage Common Collector (Chân cấp nguồn dương)
- Wi-Fi: Wireless Fidelity (Hệ thống mạng cục bộ không dây)

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành thầy Võ Việt Dũng đã tận tình giảng dạy và định hướng cho em trong suốt quá trình học tập. Nhờ những kiến thức quý báu từ thầy, đặc biệt là tư duy phát triển dự án chuyên nghiệp với VS Code, PlatformIO và quản lý mã nguồn trên Git, em đã có thể hoàn thành đề tài này một cách trọn vẹn và sát với thực tế doanh nghiệp. Dù đã rất cố gắng, nhưng bài báo cáo khó tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của thầy để đề tài được hoàn thiện hơn.

PHẦN 1: MỞ ĐẦU

1.1. Bối cảnh và tầm quan trọng của đề tài

Trong kỷ nguyên Cách mạng Công nghiệp 4.0, sự bùng nổ của Internet of Things (IoT - Internet vạn vật) đã và đang làm thay đổi hoàn toàn cách con người tương tác với các thiết bị điện tử. Các hệ thống nhúng ngày nay không còn hoạt động độc lập mà đã được kết nối liên tục với mạng lưới toàn cầu để trao đổi dữ liệu. Trong bối cảnh đó, nhu cầu giám sát các thông số môi trường (như nhiệt độ, độ ẩm) và cập nhật thời gian thực một cách chính xác tuyệt đối trở nên vô cùng quan trọng đối với nhiều lĩnh vực: từ nông nghiệp thông minh, quản lý kho bãi, phòng máy chủ cho đến hệ thống nhà thông minh (Smart Home). Việc ứng dụng các vi điều khiển có khả năng kết nối Wi-Fi mạnh mẽ để giải quyết các bài toán trên là xu hướng tất yếu và mang lại giá trị thực tiễn to lớn.

1.2. Lý do chọn đề tài

Các thiết bị đồng hồ điện tử và đo nhiệt độ truyền thống thường gặp hai hạn chế lớn: (1) thời gian bị sai lệch sau một quá trình hoạt động do linh kiện dao động nội không chuẩn xác, và (2) người dùng phải quan sát trực tiếp tại nơi đặt thiết bị. Để giải quyết những hạn chế này, em quyết định chọn đề tài: **"Xây dựng đồng hồ thời gian thực với ESP32, đồng bộ NTP và giám sát thời tiết qua Telegram"**. Đề tài sử dụng vi điều khiển ESP32 – một dòng chip mạnh mẽ, giá thành rẻ và tích hợp sẵn Wi-Fi/Bluetooth. Việc kết hợp ESP32 với giao thức đồng bộ thời gian mạng (NTP) giúp đồng hồ luôn chính xác đến từng giây. Đồng thời, việc tận dụng nền tảng Telegram Bot để gửi cảnh báo từ xa giúp hệ thống trở nên linh hoạt, người dùng có thể nắm bắt tình trạng môi trường ở bất kỳ đâu chỉ với một chiếc điện thoại di động.

1.3. Mục tiêu và phạm vi

1.3.1. Mục tiêu đề tài:

- Xây dựng thành công hệ thống phần cứng bao gồm: Vi điều khiển ESP32, màn hình OLED SSD1306 và cảm biến nhiệt độ - độ ẩm DHT22.
- Lập trình cho ESP32 kết nối Wi-Fi, tự động lấy và cập nhật thời gian từ máy chủ chuẩn quốc tế (NTP Server).
- Hiển thị trực quan các thông số thời gian, nhiệt độ, độ ẩm lên màn hình OLED.
- Xây dựng kịch bản tự động đóng gói dữ liệu và gửi thông báo định kỳ qua API của Telegram Bot.
- Áp dụng quy trình phát triển phần mềm chuyên nghiệp: Sử dụng môi trường VS Code (PlatformIO) và quản lý phiên bản mã nguồn bằng Git.

1.3.2 Phạm vi nghiên cứu:

Đề tài tập trung vào việc thiết kế sơ đồ nguyên lý, lập trình bằng ngôn ngữ C++ và kiểm thử hệ thống trên nền tảng mô phỏng Wokwi, chưa đi sâu vào việc thi công bo mạch in (PCB) hay thiết kế vỏ hộp thương mại.

1.4. Ý nghĩa của đề tài

-Ý nghĩa học tập: Đề tài giúp củng cố và tổng hợp các kiến thức đã học về vi điều khiển, kỹ thuật lập trình nhúng, giao thức mạng (Wi-Fi, HTTP/HTTPS, API) và chuẩn giao tiếp ngoại vi (I2C, 1-Wire). Đồng thời, việc làm quen với VS Code, PlatformIO và Git giúp nâng cao kỹ năng làm việc thực tế, tiệm cận với yêu cầu của doanh nghiệp.

-Ý nghĩa thực tiễn: Hệ thống là một nguyên mẫu (prototype) hoàn chỉnh, có thể dễ dàng mở rộng và ứng dụng vào thực tế như: làm đồng hồ treo tường thông minh, trạm quan trắc thời tiết mini cho gia đình, hoặc hệ thống cảnh báo nhiệt độ tự động cho các vườn ươm, phòng server.

PHẦN 2: PHẦN NỘI DUNG

2.1. Nội dung của đề tài

2.1.1 Tổng quan về hệ thống:

Hệ thống là một thiết bị IoT tích hợp đa chức năng: theo dõi thời gian thực, đo lường thông số môi trường và gửi cảnh báo từ xa. Thiết bị sử dụng vi điều khiển trung tâm là ESP32 có khả

năng kết nối Wi-Fi. Thay vì sử dụng mạch thời gian thực (RTC) rời, hệ thống đồng bộ thời gian trực tiếp từ máy chủ Internet để đảm bảo độ chính xác tuyệt đối. Các thông số thời gian, nhiệt độ và độ ẩm được hiển thị tại chỗ qua màn hình OLED và tự động báo cáo qua ứng dụng Telegram trên điện thoại người dùng .

2.1.2. Thiết kế hệ thống:

2.1.2.1 Sơ đồ logic:

Hệ thống hoạt động dựa trên mô hình xử lý 3 khối cơ bản:

- Khối Đầu vào (Input): Gồm cảm biến DHT22 (thu thập nhiệt độ, độ ẩm) và tín hiệu mạng từ NTP Server (cung cấp dữ liệu ngày giờ chuẩn).
- Khối Xử lý (Processing): Vi điều khiển ESP32 tiếp nhận dữ liệu đầu vào, xử lý định dạng chuỗi, quản lý vòng lặp thời gian và điều khiển giao tiếp mạng.
- Khối Đầu ra (Output): Gồm màn hình OLED SSD1306 (hiển thị dữ liệu cục bộ) và API của Telegram Bot (truyền dữ liệu lên đám mây và thông báo cho người dùng).

2.1.2.2. Các linh kiện và chức năng:

- **Module ESP32:** Đóng vai trò là bộ não của hệ thống. ESP32 sở hữu vi xử lý lõi kép mạnh mẽ, tích hợp sẵn module Wi-Fi và Bluetooth, bộ nhớ lớn, rất phù hợp cho các ứng dụng IoT yêu cầu kết nối mạng liên tục và xử lý HTTP Request.

- **Cảm biến DHT22:** Là cảm biến môi trường giá rẻ nhưng có độ tin cậy cao, dải đo nhiệt độ từ -40°C đến 80°C và độ ẩm từ 0% đến 100%. Giao tiếp với ESP32 qua chuẩn 1-Wire (1 dây tín hiệu).

- **Màn hình OLED SSD1306 (0.96 inch):** Sử dụng công nghệ diode phát quang hữu cơ, không cần đèn nền, tiết kiệm năng lượng và cho góc nhìn rộng. Màn hình giúp người dùng quan sát thông số trực tiếp mà không cần mở điện thoại.

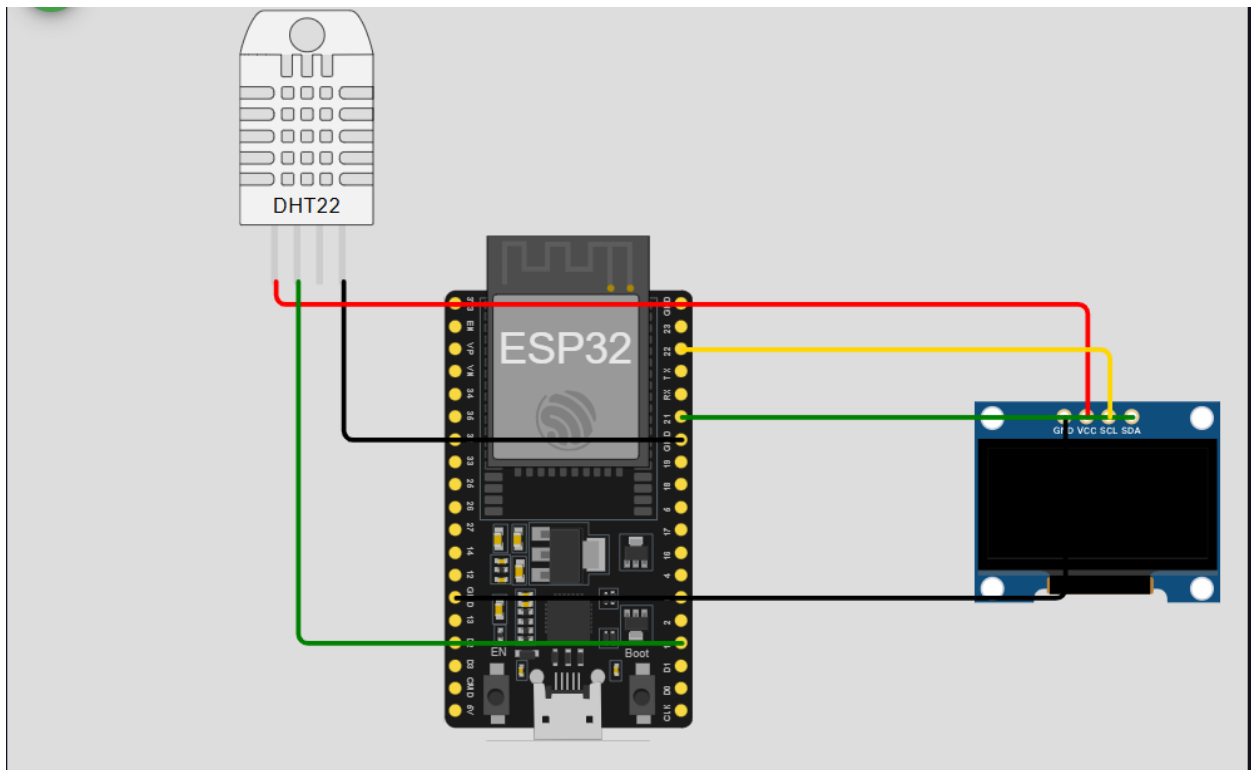
2.1.2.3. Nguyên lý hoạt động:

Sau khi được cấp nguồn, ESP32 khởi tạo các ngoại vi (OLED, DHT22) và tiến hành dò tìm, kết nối với mạng Wi-Fi đã cấu hình. Khi có Internet, vi điều khiển gửi yêu cầu (Request) đến máy chủ pool.ntp.org để lấy dữ liệu thời gian thực và cộng thêm độ lệch múi giờ (GMT+7). Trong quá trình hoạt động, hệ thống liên tục lấy mẫu nhiệt độ, độ ẩm từ DHT22 và cập nhật lên màn hình OLED cùng với thời gian (Giờ:Phút:Giây). Đồng thời, một bộ đếm thời gian ngầm được chạy; cứ mỗi khi đồng hồ chuyển sang đầu phút mới (giây 00), ESP32 sẽ đóng gói các thông số hiện tại thành một thông điệp và gửi lệnh HTTP GET qua giao thức bảo mật (HTTPS) tới máy chủ Telegram để chuyển tiếp đến điện thoại người dùng.

2.1.2.4. Sơ đồ mạch:

Quá trình kết nối phần cứng được mô phỏng chi tiết như sau:

- DHT22: Chân VCC và GND nối với 3.3V và GND của ESP32. Chân tín hiệu (Data) kết nối với GPIO15.
- OLED SSD1306: Nguồn cấp tương tự (3.3V, GND). Chân SCL (xung nhịp) kết nối với GPIO22 và chân SDA (dữ liệu) kết nối với GPIO21 theo đúng chuẩn I2C mặc định của ESP32.



(Ảnh 1: Hình ảnh sơ đồ mạch Wokwi)

2.1.3. Tổng quan về công nghệ liên quan

2.1.3.1. Giao thức NTP (Network Time Protocol)

NTP là một giao thức mạng dùng để đồng bộ đồng hồ của các hệ thống máy tính qua mạng dữ liệu chuyển mạch gói với độ trễ biến đổi. NTP hoạt động ở tầng Application (Tầng ứng dụng) của mô hình OSI, sử dụng cổng UDP 123. Dự án sử dụng pool.ntp.org, một cụm máy chủ thời

gian công cộng lớn nhất thế giới, giúp thiết bị luôn có được thời gian chuẩn xác mà không cần phân cứng RTC.

2.1.3.2. Giao thức I2C (Inter-Integrated Circuit)

I2C là một giao thức giao tiếp nối tiếp đồng bộ, đa thiết bị chủ - tớ (multi-master, multi-slave), được phát triển bởi Philips Semiconductor. Điểm mạnh của I2C là chỉ cần 2 dây dẫn (SDA cho dữ liệu và SCL cho xung nhịp) để kết nối hàng chục thiết bị ngoại vi với vi điều khiển. Trong đề tài này, OLED SSD1306 hoạt động như một thiết bị Tớ (Slave) với địa chỉ định danh là 0x3C.

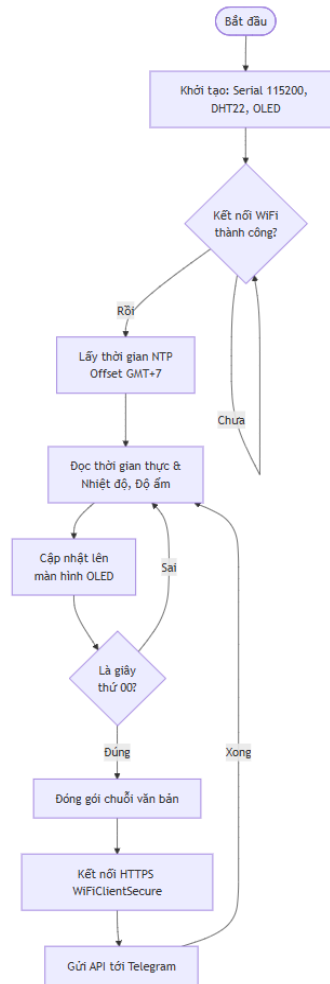
2.1.3.3. Telegram API

Telegram cung cấp một hệ thống API (Application Programming Interface) mã nguồn mở rất mạnh mẽ cho phép các nhà phát triển tạo ra các "Bot" (tài khoản tự động). Bằng cách sử dụng phương thức HTTP GET/POST kết hợp với BOT_TOKEN (chìa khóa định danh của Bot) và CHAT_ID (mã định danh của người nhận), ESP32 có thể dễ dàng ra lệnh cho máy chủ Telegram gửi tin nhắn văn bản về ứng dụng của người dùng một cách bảo mật qua chuẩn HTTPS.

2.1.4. So sánh với các giải pháp khác

- **Đồng bộ thời gian:** So với việc dùng mạch RTC (như DS3231 có pin CMOS), giải pháp dùng NTP trên ESP32 giúp tiết kiệm chi phí linh kiện, thu gọn diện tích mạch và không bao giờ bị trễ giờ qua thời gian. Tuy nhiên, điểm yếu là bắt buộc thiết bị phải có kết nối Wi-Fi.
- **Hệ thống cảnh báo:** So với việc gửi tin nhắn SMS truyền thống (cần module SIM như SIM800L), sử dụng Telegram API hoàn toàn miễn phí, tốc độ cao hơn, không phụ thuộc vào sóng viễn thông (chỉ cần Wi-Fi) và dễ dàng triển khai. Tuy nhiên, nó yêu cầu người nhận phải cài đặt ứng dụng Telegram và có kết nối mạng di động

2.2. Cài đặt hệ thống



(Ảnh 2: Sơ đồ khối flowchart minh họa)

Môi trường phát triển và Quản lý mã nguồn: Hệ thống không sử dụng Arduino IDE cơ bản mà được triển khai trên nền tảng **PlatformIO** tích hợp trong **Visual Studio Code (VS Code)**. Cấu trúc dự án tuân thủ tiêu chuẩn C/C++ với file cấu hình platformio.ini để tự động hóa việc quản lý các thư viện phụ thuộc (như Adafruit_GFX, Adafruit_SSD1306, DHTesp). Đồng thời, mã nguồn được khởi tạo kho lưu trữ (repository) cục bộ và đẩy lên hệ thống quản lý phiên bản Git (thông qua các lệnh git init, git commit, git push), giúp kiểm soát các thay đổi và dễ dàng làm việc nhóm, tiệm cận với quy trình thực tế của doanh nghiệp phần mềm.

Phân tích logic mã nguồn cốt lõi:

- Hàm setup(): Thực hiện kết nối mạng cục bộ. Sau khi lấy được IP, hệ thống gọi cấu hình thời gian qua hàm configTime(gmtOffset_sec, daylightOffset_sec, ntpServer).
- Hàm loop(): Cập nhật liên tục cấu trúc timeinfo và đọc giá trị cảm biến. Nếu thỏa mãn điều kiện chu kỳ (giây hiện tại bằng 00 và cách lần gửi trước đủ lâu), hàm sendTelegramMessage() sẽ được gọi.

- Hàm xử lý HTTPS: Sử dụng WiFiClientSecure và thiết lập client.setInsecure() để bỏ qua việc xác thực chứng chỉ SSL phức tạp, giúp ESP32 kết nối trực tiếp đến API của Telegram và thực hiện gói lệnh HTTP GET chứa các tham số nhiệt độ, độ ẩm.

2.3. Kết quả thực hiện

Dự án đã được mô phỏng hoàn thiện trên nền tảng Wokwi và đáp ứng toàn bộ các yêu cầu đặt ra:

- **Về hiển thị:** Giao tiếp I2C hoạt động ổn định, màn hình OLED hiển thị rõ ràng, tốc độ làm mới màn hình đáp ứng tốt tính năng đồng hồ đếm giây.
- **Về cảm biến:** Các giá trị nhiệt độ, độ ẩm phản hồi tức thời lên màn hình khi có sự tinh chỉnh thông số từ thanh trượt mô phỏng của môi trường Wokwi.
- **Về hệ thống cảnh báo:** Mạch kết nối Wi-Fi giả lập thành công, bắt được gói tin NTP. Các tin nhắn báo cáo định kỳ được đẩy về điện thoại qua Bot Telegram chính xác vào đầu mỗi phút mà không có độ trễ hay mất mát dữ liệu mạng



(Ảnh 3 : Hình ảnh chụp màn hình tin nhắn Telegram trên điện thoại)

PHẦN 3: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

3.1. Kết luận.

Đề tài "**Xây dựng đồng hồ thời gian thực với ESP32, đồng bộ NTP và giám sát thời tiết qua Telegram**" đã được thực hiện thành công và bám sát với các mục tiêu đề ra ban đầu. Thay vì sử dụng các phương pháp lập trình truyền thống trên Arduino IDE, dự án đã tiếp cận theo hướng kỹ thuật phần mềm chuyên nghiệp: Lập trình C/C++ trên nền tảng Visual Studio Code (PlatformIO), mô phỏng phần cứng trực quan trên Wokwi và quản lý tiến trình phát triển mã nguồn bằng hệ thống Git. Hệ thống không chỉ giải quyết được bài toán đồng bộ thời gian chuẩn xác qua Internet mà còn ứng dụng linh hoạt các giao thức mạng (HTTP/HTTPS, API) để biến một mạch điện tử thông thường thành một thiết bị IoT có khả năng tương tác với người dùng từ xa.

3.2. Nhận xét

Trong quá trình thực hiện và chạy thử nghiệm hệ thống, em rút ra một số nhận xét sau:

- **Về mặt phần cứng (mô phỏng):** ESP32 thể hiện khả năng xử lý đa nhiệm rất tốt. Dù phải gánh vác việc duy trì kết nối Wi-Fi, chạy bộ đếm thời gian, quét cảm biến và điều khiển màn hình I2C liên tục, hệ thống vẫn không xảy ra hiện tượng tràn bộ nhớ hay treo vi điều khiển (Watchdog Timer reset).
- **Về mặt phần mềm:** Việc chuyển đổi từ môi trường Arduino (.ino) sang C++ chuẩn (.cpp) trên PlatformIO đòi hỏi tính tuân thủ quy tắc lập trình cao hơn (như phải khai báo thư viện <Arduino.h>, khai báo nguyên mẫu hàm trước khi sử dụng). Tuy nhiên, điều này giúp cấu trúc mã nguồn trở nên chặt chẽ, dễ bảo trì và mở rộng hơn rất nhiều.
- **Về tích hợp API:** Giải pháp dùng `client.setInsecure()` để bỏ qua việc xác thực chứng chỉ số (Certificate) giúp mạch dễ dàng kết nối với máy chủ HTTPS của Telegram. Tuy nhiên, điều này phù hợp cho mô hình thử nghiệm (prototype) hơn là một sản phẩm thương mại yêu cầu bảo mật cao.

3.3. Đánh giá

Ưu điểm của hệ thống:

- Hoạt động ổn định, độ chính xác của đồng hồ là tuyệt đối do được đồng bộ liên tục từ máy chủ quốc tế.
- Chi phí linh kiện thấp (nếu triển khai phần cứng thật), dễ dàng tìm kiếm và thay thế.
- Tính năng cảnh báo qua Telegram hoàn toàn miễn phí, có tính ứng dụng cao, giúp người dùng nắm bắt thông tin mọi lúc mọi nơi mà không cần cài đặt thêm các ứng dụng IoT phức tạp của bên thứ ba.

Hạn chế của hệ thống:

- Mức độ phụ thuộc cao vào mạng Internet: Nếu mất kết nối Wi-Fi, đồng hồ không thể lấy được giờ và tính năng gửi tin nhắn cũng bị vô hiệu hóa.
- Do chạy trên môi trường mô phỏng Wokwi, hệ thống chưa được đánh giá chính xác về khả năng tiêu thụ điện năng thực tế, cũng như tác động của nhiễu tín hiệu vật lý đến đường truyền I2C và 1-Wire.

3.4. Kiến nghị

Để hệ thống hoàn thiện hơn và có thể áp dụng vào đời sống thực tế, em đề xuất một số hướng phát triển sau:

- **Khắc phục nhược điểm mất mạng:** Tích hợp thêm một module thời gian thực (RTC DS3231) vào mạch cứng. ESP32 sẽ ưu tiên lấy giờ từ NTP; trong trường hợp rớt mạng Wi-Fi, mạch sẽ tự động chuyển sang đọc giờ từ DS3231 để đảm bảo hiển thị không bị gián đoạn.
- **Tối ưu hóa năng lượng:** Thiết kế mạch phần cứng thực tế và ứng dụng các thuật toán ngủ sâu (Deep Sleep) cho ESP32. Chỉ khi đến chu kỳ báo cáo hoặc khi nhiệt độ/độ ẩm vượt ngưỡng cài đặt (ví dụ $>35^{\circ}\text{C}$), vi điều khiển mới thức dậy, kết nối Wi-Fi và gửi cảnh báo, sau đó lại ngủ tiếp để tiết kiệm pin.
- **Đóng gói sản phẩm:** Tiến hành vẽ mạch nguyên lý, thiết kế mạch in (PCB) trên phần mềm Altium hoặc Proteus, sau đó thi công phần cứng và thiết kế vỏ hộp in 3D để tạo ra một sản phẩm hoàn chỉnh, thẩm mỹ và có thể thương mại hóa.

TÀI LIỆU THAM KHẢO

1. Espressif Systems. (2023). *ESP32 Series Datasheet*. Truy cập từ tài liệu kỹ thuật của nhà sản xuất.
2. Adafruit Industries. *Adafruit GFX Graphics Library & Adafruit SSD1306 Library Documentation*. GitHub Repository.
3. Telegram. *Telegram Bot API - HTTP-based interface created for developers building bots for Telegram*. Truy cập từ: <https://core.telegram.org/bots/api>
4. PlatformIO Labs. *PlatformIO Documentation - Professional collaborative platform for embedded development*. Truy cập từ: <https://docs.platformio.org/>
5. Wokwi. *Wokwi Arduino and ESP32 Simulator Documentation*. Truy cập từ: <https://docs.wokwi.com/>

